Subclass Ticket Access

Practice Problem 2

Concept and approach

Extend the classifier to the two cross-package subclass contexts. Only protected is allowed for SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE; a protected member is denied when accessed through a parent-typed reference in the PARENT_TYPE context.

Important implementation notes

The important distinction is the expression's declared type. The fact that the referenced object is a PremiumMovieTicket does not make access through a MovieTicket-typed expression legal across packages.

Java solution

```
class AccessChecker {
static String classifyAccess(String modifier, String context) {
switch (modifier) {
case "private": return context.equals("SAME_CLASS") ? "ALLOWED" : "DENIED";
case "default": return context.equals("SAME_CLASS") || context.equals("SAME_PACKAGE") ? "ALLOWED" : "DENIED";
case "protected":
return context.equals("SAME_CLASS") || context.equals("SAME_PACKAGE") ||
context.equals("SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE") ? "ALLOWED" : "DENIED";
case "public": return "ALLOWED";
default: throw new IllegalArgumentException("Unknown modifier: " + modifier);
}
}
}

// In a different package:
class PremiumMovieTicket extends MovieTicket {
double loyaltyPrice() {
return this.ticketPrice; // legal: this is the subclass type
}
double parentTypedAccess(MovieTicket ticket) {
// return ticket.ticketPrice; // compile-time error for protected access
return this.ticketPrice;
}
}
```

Sample output

```
ALLOWED
DENIED
```